

Mulligan Stew FAQ

Response by the DAE team to Pat Helland's Questions

VO.8 02025-Jul-24

Pat Helland Questions

1. **What is the specific problem space you're trying to solve?**
2. **What are the characteristics of the participants in the communication?**
3. **What is the "thing" being transacted?**
4. **What does atomicity mean in your system?**
5. **How do you define the endpoints of communication?**
6. **What are the participants in the transaction?**
7. **Primary frustration:** the discussion lacked concrete definitions and kept shifting between different domains without clearly defining the core problem. He repeatedly asked for a bounded, specific explanation of what the team is trying to achieve with atomicity in network communications.

Paul

Maintaining liveness and synchronizing processes in networks is a challenge when packets can be dropped, re-ordered, duplicated or delayed

- Conventional networks require protocol stacks and applications to use timeouts and retries to maintain liveness
- This makes exactly-once semantics impossible and precipitates retry storms which lead to unbounded tail latency and transaction failure
- This results in silent corruption of data structures and loss of data — A problem seen in every distributed database for decades
- These failures are not understood because customers (under NDA) may not publish results that embarrass vendors

Definitions of Atomicity

The term atomic is used in various contexts in CS, often meaning indivisible or uninterruptible. For example:

1. **Atomic Operations:** Indivisible instructions at CPU or memory level.
2. **Atomic Transactions:** Database or distributed transactions that commit or rollback as a unit.
3. **Atomic Packets:** Packets that cannot be fragmented (e.g., Ethernet packets with the Don't Fragment bit set).
4. **Atomic Updates:** File systems or memory updates that never leave partial state visible.
5. **Atomicity in Distributed Systems:** Ensures actions across multiple nodes appear as a single event.
6. **Atomic Broadcast:** Either all intended recipients receive a message or none do.
7. **Dissemination Atomicity** is what unreliable networks do today, flaw: assumes a global simultaneity plane.
8. **Synchronization Atomicity** requires coordination. Common knowledge synchronized at both endpoints.

Ethernet "appears" atomic only at the frame level. Reliable delivery across frames requires acknowledgments, either sequentially (alternation/verify bits) or through pipelined mechanisms like credit-based flow control and RTS/CTS. GNet in Aquila combines these ideas with a modern high-performance design suitable for datacenter networks.

The impossibility of achieving Token/Pebble Atomicity without Reversibility.

Ethernet Is not ACID or Token Atomic, In the sense of Charlie Bennet's pebbles¹:

"reversible machines can simulate ordinary ones in quadratic space. These results refer to reversible machines that save their input, thereby insuring a global 1:1 relation between initial and final IDs, even when the function being computed is many-to-one. Reversible machines that instead erase their input can of course compute only 1:1 partial recursive functions and indeed provide a G6del numbering of such functions"

The DAE/OAE protocol provides atomic token transfer with a shared state machine—isomorphic to the the No-Cloning/No-Broadcasting Theorems.

This is what enables us to guarantee exactly-once semantics to distributed applications and databases.

¹ Charles H. Bennett. "TIME/SPACE TRADE-OFFS FOR REVERSIBLE COMPUTATION". https://mathweb.ucsd.edu/~sbuss/CourseWeb/Math268_2013W/Bennett_Tradeoffs.pdf

Sahas

1. What is the specific problem space you're trying to solve?

Over the past two decades, application infrastructure has scaled to planetary levels: serving billions of users, moving trillions of dollars, and underpinning everything from logistics to governance. Thanks to the miracles of datacenter engineering and cloud architecture, we enjoy 99.99% uptime across the globe. It's one of the industrial age's greatest feats, enabled by practically infinite money.

This rapid scale out has left a major opportunity in reducing the cost of infallibility --- in the physical infrastructure deployed by the rack and in the development, deployment, and operation of applications that run on it.

DÆDÆLUS is rethinking how networks serve distributed computing. While compute has scaled exponentially, the networks supporting it continue the legacy of decisions made half a century ago, when bandwidth was scarce and the network was passive.

2. What are the characteristics of the participants in the communication?

Communication on our network is fundamentally an interaction of two nodes: the sender and receiver separated by a single cable. The node, a fungible member of the network, assumes the role of sender or receiver depending on the arrow of causality that defines the interaction.

Path-cast and tree-cast communication emerge compositionally from these neighbor-to-neighbor interactions, enabling end-to-end application messages to reach single or multiple destinations. The endpoints are not named by IP addresses, but by communication trees which heal locally and are inherently virtualized over the physical topology. IP addresses can be requested after the fact, for transport-layer compatibility.

3. What is the "thing" being transacted?

Application-send is an event in spacetime, tagged with the message of interest for the destination. Knowledge of this event and its message must be handled with care on the network and deterministically delivered or returned to sender.

This event carries semantic weight in that it represents the application's intention to act at a distance. What is transacted is not the packet itself, but knowledge that a particular message was sent, received, and committed into the causal history of the receiver. This knowledge must propagate deterministically, without duplication, loss, or ambiguity. If the network fails to deliver that knowledge, it must return it to the sender with certainty.

4. What does atomicity mean in your system?

Every message crossing a physical link is part of a transaction: a clearly bounded interaction with two possible outcomes:

- Accepted and acknowledged at the next hop

- Reversed cleanly back to the sender if delivery fails.

At no point is a message duplicated, partially delivered, or ambiguously dropped. There is no retry storm, no exponential backoff, no hidden nondeterminism. Messages move forward or return, never disappear.

This atomicity is enforced by the passive coordinator of the link, which ensures consistency of state across both sides of the interaction. Because these atomic transactions compose across multiple hops, application level delivery emerges as a chain of causally complete events, not a best-effort relay.

5. How do you define the endpoints of communication?

Endpoints in DÆDÆLUS are not addresses but positions in a communication structure.

In the traditional model, the endpoint is an IP address: an identity assumed to be stable, reachable, and globally meaningful. In contrast, our endpoints are leaves or interior nodes in a self-healing communication tree, identified by their relationship to the message's path, not by their global coordinates.

These endpoints are dynamic---they can migrate, split, or merge, while preserving application semantics. Addressing is ephemeral, relative, and resolved only at the moment of need. This makes the system naturally resistant to failure, congestion, or reconfiguration, without the need for global routing tables or convergence delays.

6. What are the participants in the transaction?

Every transaction in Atomic Ethernet is enacted by a pair of directly connected nodes, the sender and receiver, across a single physical link. These two are the active participants in the transaction: the sender initiates causality by attempting to transmit a message, and the receiver confirms its successful and ordered receipt.

The third entity in link-level transaction is the link-layer coordinator: a passive monitor embedded in the fabric of the network itself. It does not initiate or process messages, but enforces atomicity: ensuring that messages are either accepted and acknowledged or reversibly returned, never duplicated or lost. Its presence reinforces the fate of every message in flight and guarantees link-level consistency.

Atop this fabric-level coordination lies a higher-level application coordinator. This entity tracks the liveness and progress of multi-hop, end-to-end transactions composed from many link-local atomics. It does not enforce delivery--but ensures the system does not stall. Its job is to name deadlocks, detect non-progress, and orchestrate recovery or re-routing at the application granularity.

David

1. What is the specific problem space you're trying to solve?

- Data center down to chiplet and wafer scale L2 networking. Efficiency, determinacy and security thereof.
- With an eye on the context of 802.3/IP networks today have nondeterministic packet delivery with well described defects: packet duplication, packet loss, packet reordering.

2. What are the characteristics of the participants in the communication?

This is a L2 network analogous to 802.3. The participants are L2 network nodes and higher layers sending IP packets back and forth.

3. What is the "thing" being transacted?

- Packets being presented at the MSDU interface.
- Packets generated at L2 by L2 nodes to manage the network

4. What does atomicity mean in your system?

- Delivery of a packet once or never.
- In a successful transaction, (1) A transfer has happened (2) The sender knows the transfer happened and (3) the receiver got the transfer and knows the sender knows the receiver got it.
- In a failed transaction (1) the sender knows is failed. (2) the receiver either knows nothing or knows the transaction failed

5. How do you define the endpoints of communication?

- MSDU/BPDU to MSDU/BPDU
- L2 MAC to L2 MAC (for network local management)
- IP service endpoints. Either
 - Transaction aware IP based protocol thing or
 - Conventional IP protocol like TCP and others.

6. What are the participants in the transaction?

- L2 MAC with L2 mac and intermediate nodes on the path and other nodes not on the path that are involved in topology learning.
- (a) plus: IP based transaction aware protocol thing at both ends.

7. Primary Frustration

His primary frustration was that the discussion lacked concrete definitions and kept shifting between different domains without clearly defining the core problem. He repeatedly asked for a bounded, specific explanation of

what the team is trying to achieve with atomicity in network communications.

8. Security? WTH?

I'll add my own - security that addresses the concerns of L2 insecurity in wired networks.

Counterexamples

- 802.16 PKMv2 - addresses the concerns of theft of service, not the needs of customer privacy.
- 802.11i - addresses the wireless link security but not the onward traffic.
- 802.3 MACSEC. No practical auth without 802.1X infrastructure and centralized identity management.

Steve

1. What is the specific problem space you're trying to solve?

Imagine a distributed application, say a distributed database or distributed storage, running within a single fault containment zone of a data center. Within that one distributed data base deployment, the instances running on various computers need to continuously be in agreement on who's doing what and on their collective persistent data. Although each distributed application chooses from a variety of ways to achieve this continuous agreement, what they all have in common is the exchange of messages roughly in the pattern of consensus algorithms between those software instances on different computers, where loss or delay of enough of these messages leads to "tail" like behavior such as performance collapse, an application crash, or worse, silent corruption of data belonging to the distributed application.

At present, distributed applications have an unsolved fundamental "design assurance" problem ensuring no combination of delayed or lost-pending-timeout packets will cause a hang, crash, or data corruption.

We are trying to create a local network with three properties:

- The network routes packets around failed links, without dropping packets. The sender and receiver can together observe additional sub 10 microsecond delay during this route around.
- Knowledge of in process packets is confined to the link; the receiver never learns any information about failed messages; the sender simply learns "the receiver has it" or "couldn't get the message there" within sharply bounded time (say a microsecond). Corner cases a distributed app has to deal with today are designed out.
- The endpoints (the sender and receiver) are application code in user space. One endpoint of a communication session can be moved to a different server without the other endpoint knowing, other than observing delay. All involved software and hardware components below the application must deliver these

properties, which current network stacks, drivers, and NICs do not.

We believe solving this assurance of consensus communication problem enables distributed applications to run with >10x more instances on >10x cheaper smaller computers, in applications that don't have a compelling scale economy in memory size. Xeons are far into diminishing returns for performance/cost.

- a. The protocol should run over not just leading edge PHY serdes speeds, but also over old cheap point to point links. It should be equally possible to make a \$1000 connection between leading edge AI chips in a water cooled mesh, and a \$1 connection 1cm long between adjacent chiplet modules. [Editorial note: I think there's fame in the former, and profit changing the economics of the computer business in the latter]

Note that today's networks are generally optimized for bandwidth. This network is optimized for latency. To borrow from an earlier era, Seymour Cray's "Don't hide latency, eliminate it!"

2. What are the characteristics of the participants in the communication?

All communications are sessions between two participants. Reliable multicast is achieved by intermediary/intermediaries which unicast to each destination in the multicast set of participants/endpoints. There is no such thing as a broadcast.

A participant is a software endpoint in user space belonging to an instance of a distributed application. That participant has a "name", not a geographic address. Knowledge of a "name" is granted during creation of the instance.

For backward compatibility, an IP address can be bound to a "name" in this network in much the same way it was bound to a MAC address in the networks of 20 years ago.

3. What is the "thing" being transacted?

The fundamental operation of sending a message in this network has ACID properties, with delivery of the message considered stateful. "Exactly once" is part of those ACID properties, but not all.

More complex primitives can be built in the hardware, and more complex relationships between endpoints built in the control plane software (which is very different from today's network control planes). These could radically reduce the latency of application level transaction like operations, such as consensus around data base subtransactions and in particular the "commit". It is expected these will be proprietary extensions to the standard for now; useful primitives proven to be broadly useful in reality can be brought into the standard later.

4. What does atomicity mean in your system?

For a message: either the message was delivered to the receiving software endpoint and both sender and receiver know this occurred, or all traces of its existence are gone and only the sender remembers the attempt.

For a data item shared between two nodes: updates to that data item, in hardware, as a primitive, lock the software endpoints at both ends of the link out of accessing the data item, update the shared value in both locations, and then unlock access at both ends. "Lock free" implementation of this is preferred.

This can be extrapolated to a value shared between "N" nodes; many implementations with the necessary properties are possible; a single standard is not (yet) proposed.

5. How do you define the endpoints of communication?

Endpoints are analogous to InfiniBand (including RoCE) queue pairs in user space.

Native behavior is not addressing a message to a NIC, which hands it to a VM, which hands it to a container, which selects an application destination. The message is addressed to the application's queue. This concept has been around since the research which led to InfiniBand.

A connection between two endpoints is a session analogous to a TCP socket(); instead of TCP's various behaviors, the ability to communicate with a target is by whitelist permission and forwarding tables in the NICs of endpoints are populated either based on that very sparse permission or on the actual opening of a session. The communication is interlocked per message at least per hop (I propose that it be interlocked end to end but the proposal is nuanced and very different.)

Each endpoint has a "name" which can be relocated independently of all other endpoints (ie this is decidedly not IP route summarization and all the complexity that imposes on Kubernetes). No packets are lost during endpoint relocation, but no standard is yet proposed for the whole of distributed application instance relocation, which is much more than just moving the network endpoint.

6. What are the participants in the transaction?

In its simplest form, a transaction is a single message, delivered exactly once with ACID properties, such that it does not complete visible to user space software until both ends know the communication succeeded.

But I think this question is about much more complex transactions which span persistent data located in multiple instances on different computers, must have ACID properties, and proceed in parallel with other transactions. The intent here is to provide primitives, in hardware, at the endpoints, which can acquire and release suitably granular locks, and update hardware owned values, at link latency rather than software or storage latency. That is, to grab the knob of how many transactions a database has in process at a time, and turn that knob very hard toward making an individual transaction so fast that very few actually run in parallel. Seymour Cray and latency, again.

7. Primary Frustration

His primary frustration was that the discussion lacked concrete definitions and kept shifting between different domains without clearly defining the core problem. He repeatedly asked for a bounded, specific explanation of

what the team is trying to achieve with atomicity in network communications.

I have shared that frustration over the last 5 years and hope we can start writing down concrete data structures and algorithms acting on those data structures, for the distributed system which is our network.

Additional thoughts beyond Pat's Question

I left one piece out of my "what is the specific problem you're trying to solve" because I thought it was too far afield of what Pat Helland needed. At the end of that first section:

- b. Because these are point to point links, there is a future opportunity. Today all of the energy used to send a bit down a link (and it's a lot) is dissipated. In the future it will be possible to reuse some of that energy (second law of thermodynamics says not all) to send a bit back. Likewise, the research topic of "reversible" logic offers us a way to reuse energy used to process bits once they arrived at the receiver, and in the process ensure there is no memory (and therefore no side effects) of a failed arrival. This is not a first release (or even second release) topic, but rather a future we simply need to allow individual implementations to choose.

Reducing the CapEx cost of distributed computation, and the energy cost of distributed computation, are a really, really big deal for the world, a grand challenge for this century. —Steve

Alan

Pat's choice of word "atomicity" instead of "atomic" leads me to believe he's using the word in the database sense, which is not what you mean. Here's the way I explain it.

The word "atomic" in the description is an analogy with that word in physics.

In a quantum system, two particles can be entangled, which means they share a wavefunction. In a conventional link, there is a state machine on each side of the link. In an "atomic" link, there is one state machine spanning the link, with the state being carried in the packets being exchanged. That means the two sides of an "atomic" link can never be inconsistent with each other.

Quantum systems are reversible. A conventional link resets its state on bit errors, leading to the possibility that one side will reset but the other won't. An "atomic" link runs the state machine backward to its initial state, guaranteeing consistency.

Another analogy is with the quantum no cloning theorem, that says you can't copy a quantum state. The analogy on the "atomic" link is that packets are never duplicated. Finally, another analogy is conservation laws. In an "atomic" network, packets are conserved, i.e., never lost.

As with any analogy, it's a way to guide your thinking, but don't push it too far.

Open Atomic Ethernet

[Consensus from many conversations with companies, groups and individuals prior to and during 2025. Special thanks to the Mulligan Stew Gang 2023-2025].

OCF [Open Atomic Ethernet](#)

Ethernet has been the cornerstone of networking for over five decades, adapting to changing technologies, applications, and economic realities. New data center and edge opportunities—such as directly linking chiplet modules in ultra-low-cost meshes, achieving extreme low latency by executing application actions at hardware speed in the network interface, and applying these techniques to distributed updates of persistent storage or to distributed, application-level transactions—push beyond the boundaries Ethernet established in an earlier era. These needs cannot be met simply by extending Ethernet's historical capabilities.

Open Atomic Ethernet will look ahead to the next 50 years of Ethernet networking. It will focus on ways to achieve higher performance, guaranteed service, lower latency, fewer errors, and greater scalability.

Scope

Align what the network does with what the distributed application needs to accomplish. Build a world where the network is not allowed to create a mess the application must discover and then clean up, but rather must either ensure both sender and receiver (user space software endpoints in the distributed application) know a message was successfully sent and received, or only the sender "instantly" knows it failed, and no other node or part of the network knows that message ever existed.

This entanglement between sender and receiver, with no library, software, or hardware permitted to disturb that entanglement, is a fundamental change from the "throw it over the wall and hope it gets there" of the last 50 years. From a different perspective, a network message becomes a transaction which exhibits ACID properties: atomicity, isolation, and durability directly, and consistency because that message exists only between endpoints and is not visible to any other observer.

Rethink Ethernet's core primitives: frame structure, error recovery, and flow control, in the context of Shannon Information Theory.

Introduce API's for programmable minimal compute capability at the NIC level to enable reversibility and support application-specific protocol behavior.

Address the trade-offs between signals, noise and energy dissipation, building upon principles from Quantum Thermodynamics.

New Fundamental multi-phase primitives: build state machines into the link itself to enable Network assisted transactions.